

ADR-004 — 지표, 거래 이력, Performance와 CSV 정책

항목	값
상태	Accepted
결정일	2026-08-20
적용 결정	D-07, D-10, D-11, D-12, D-13
사용자 날짜대	Asia/Seoul

1. 4H 지표 공식

모든 입력과 중간 계산은 Python `Decimal`을 사용한다. 계산용 local Decimal context는 `precision=34, ROUND_HALF_EVEN`이다. float 변환은 금지한다.

1.1 입력 준비

- 4H Kline을 `open_time` 오름차순으로 정렬하고 중복 key를 제거한다.
- EMA9 series와 swing은 `closed == true`인 확정봉만 사용한다.
- LR slope를 만들려면 최소 14개의 연속 확정 4H close가 필요하다. 첫 9개로 seed를 만들고 이후 5개를 적용해야 EMA9가 6개 생긴다.
- swing 판정에는 최소 2개의 확정 swing high와 2개의 확정 swing low가 추가로 필요하다. 14개가 있어도 이 네 pivot이 없으면 입력 부족이다.
- 현재가는 같은 `MarketSnapshot.version`에 들어 있는 진행 중 4H 봉의 최신 close다. 계산 도중 version이 바뀌면 결과를 버리고 새 version으로 다시 준비한다.

1.2 EMA9

```
alpha = 2 / (9 + 1) = 0.2
seed_ema9 = 첫 9개 확정 close의 산술평균
ema9_t = close_t * alpha + ema9_(t-1) * (1 - alpha)
```

series의 첫 값은 아홉 번째 확정봉에 정렬한다. 중간값은 반올림하지 않는다. wire, fixture와 trace에 기록할 때만 소수점 8자리로 `ROUND_HALF_EVEN`한다.

1.3 최근 6개 EMA9의 LR slope

최근 6개 확정 EMA9를 오래된 순서로 `y_0...y_5, x=0...5`에 놓는다.

```
x_mean = 2.5
y_mean = sum(y_i) / 6
lr_slope = sum((x_i - x_mean) * (y_i - y_mean))
            / sum((x_i - x_mean)^2)
ema9_slope = lr_slope / current_price * 100
```

단위는 % / 4H봉이다. `ema9_slope`는 소수점 8자리로 반올림한 값을 RegimeSTM guard에 전달한다. 경계 `-0.30`, `-0.15`, `0.15`, `0.30`도 Decimal로 비교한다.

1.4 swing 구조

- `left=2`, `right=2`로 고정한다.
- index `i`의 high가 좌우 각 2개 확정봉 high보다 모두 엄격히 큰 경우 swing high다. low가 좌우 각 2개 low보다 모두 엄격히 작은 경우 swing low다.
- 같은 값 tie는 pivot으로 인정하지 않는다.
- 오른쪽 2개 봉까지 확정된 pivot만 사용한다.
- 유의 변화율 threshold는 `0.30%`로 고정한다.

최근 두 확정 swing high를 `previous_high`, `latest_high`, 최근 두 low를 같은 방식으로 정의한다.

```
high_change_pct = (latest_high - previous_high) / abs(previous_high) * 100
low_change_pct  = (latest_low  - previous_low)  / abs(previous_low)  * 100

HH = high_change_pct >= 0.30
LH = high_change_pct <= -0.30
HL = low_change_pct  >= 0.30
LL = low_change_pct  <= -0.30
```

threshold 사이 값에서는 해당 high/low 방향 boolean을 모두 false로 둔다.

1.5 live EMA9

진행봉의 현재가를 확정봉 close series에 한 번만 적용한다.

```
live_ema9 = current_price * 0.2 + latest_closed_ema9 * 0.8
```

이 값은 확정 EMA9 series에 append하지 않는다. current price와 live EMA9는 같은 snapshot version과 `calculated_at`을 공유한다.

1.6 Golden vector

다음 입력을 Phase 3 fixture로 그대로 사용한다.

```
closed close = [100, 101, 102, 103, 104, 105, 106,
                107, 108, 109, 110, 111, 112, 113]
closed high  = [110, 111, 115, 112, 113, 116, 114,
                115, 117, 116, 117, 118, 117, 116]
closed low   = [ 99,  98,  97,  95,  97,  98,  96,
                99, 100, 101, 102, 103, 104, 105]
current_price = 110
```

기대 결과는 다음과 같다.

값	기대 결과
확정 EMA9 series	[104, 105, 106, 107, 108, 109]
최근 6개 OLS raw slope	1
ema9_slope	0.90909091 % / 4H봉
확정 swing high index/value	(2,115), (5,116), (11,118)
최근 high 변화	$(118 - 116) / 116 * 100 = 1.72413793\%$, HH=true
확정 swing low index/value	(3,95), (6,96)
최근 low 변화	$(96 - 95) / 95 * 100 = 1.05263158\%$, HL=true
LH, LL	false, false
live_ema9	109.20000000
Regime 결과	강상승 복합조건 충족으로 TYPE_2

2. 거래 이력 JSONL schema

2.1 파일 형식

- encoding은 UTF-8, BOM 없음, 줄바꿈은 LF다.
- 한 줄은 하나의 완결된 JSON object이며 record_type="trade"다.
- 기존 record는 schema_version=1, Phase 9 이후 일반 앱 execution은 schema_version=2를 가진다. BNB fee 근거가 있는 앱 execution은 v3, 검증된 외부 수동 매도는 v4다. reader는 네 version을 함께 읽고 기존 row의 version과 의미를 보존한다.
- 가격·수량·금액·수익률은 exponent 없는 JSON string이다. JSON number로 금융 수치를 저장하지 않는다.
- timestamp는 UTC RFC 3339 형식이며 Z를 사용한다.

schema version 1과 2는 같은 필수 key를 사용한다. key 순서는 의미에 영향을 주지 않지만 writer는 아래 순서를 사용한다. 회계 의미는 schema_version으로 구분하므로 version은 canonical Trade equality와 order ID 충돌 판정에도 포함한다.

```
{
  "schema_version": 2,
  "record_type": "trade",
  "trade_id": "trade-01",
  "order_id": "123456",
  "client_order_id": "bat-session-intent-0",
  "symbol": "ETHUSDT",
  "executed_at": "2026-08-20T11:00:00.000000Z",
  "side": "BUY",
  "regime_type": "TYPE_0",
  "strategy": "CASE_B",
  "requested_quantity": "1",
  "executed_quantity": "1",
  "executed_amount": "100",
  "average_fill_price": "100",
```

```

"market_price_at_decision": "99.9",
"fee_amount": "0.001",
"fee_asset": "ETH",
"fee_quote_amount": "0.1",
"allocated_cost_basis": null,
"realized_pnl": null,
"realized_return_rate": null,
"exit_reason": null
}

```

- BUY의 realized/allocated/exit 필드는 null이다.
- SELL은 `allocated_cost_basis`, `realized_pnl`, `realized_return_rate`를 Decimal string으로 저장하고 `exit_reason`을 기록한다.
- `fee_quote_amount`는 체결 시점에 quote asset으로 정규화한 수수료다. 원래 수수료 금액과 asset도 함께 보존한다. fee asset이 USDT이면 원래 금액과 같다. ETH이면 각 fill의 `commission * fill price`를 먼저 계산한 뒤 그 결과를 합산한다. 여러 fill의 maker/taker rate가 다를 수 있으므로 총 `fee_amount * average_fill_price`로 다시 계산하지 않는다. 제3 asset이면 임의의 시세를 사용하지 않고 `FEE_ASSET_CONVERSION_REQUIRED`로 reconciliation을 요구한다.
- Decimal string은 `^-?(0|[1-9][0-9]*) (\.[0-9]+)?$`를 만족해야 하며 NaN, Infinity, exponent와 locale separator를 허용하지 않는다.

2.2 Fee 회계 version과 migration

v1은 Phase 8에서 확정한 기존 의미를 보존한다. BUY는 fee asset과 무관하게 gross `executed_quantity`를 Position에 더하고 `executed_amount + fee_quote_amount`를 원가에 더한다. 기존 v1 row를 단순 재시작만으로 v2 공식으로 다시 해석하지 않는다.

v2는 아래 3.1의 실제 자산 흐름 공식을 사용한다. USDT-fee row는 v1과 v2의 결과가 같고, ETH-fee BUY만 net 수량과 실제 quote debit 기준으로 달라진다. `Trade.from_order_execution` 및 JSONL writer의 일반 신규 record는 v2이며, reader와 Repository는 혼합 파일의 원래 version을 그대로 보존한다. 동일 order ID의 row가 version만 다르더라도 서로 다른 canonical 내용이므로 `ORDER_HISTORY_CONFLICT`로 거부한다.

현재 열린 Position lot에 양수 ETH 수수료의 v1 BUY가 하나라도 남아 있으면 그 수량은 legacy gross 회계에 의존한다. Account에 별도 ETH가 있어 단순 잔액 비교를 통과하더라도 자동 거래를 재개하지 않고

`HISTORY_ACCOUNTING_MIGRATION_REQUIRED`로 fail closed한다. 같은 열린 lot에 v2 row가 뒤따르더라도 v2가 legacy 수량을 담아 migration 표식을 지우도록 허용하지 않고 history replay 시점에 동일 오류로 중단한다. 해당 lot이 v1 공식에 따라 이미 전량 SELL되어 닫혔다면 과거 SELL의 저장된 `allocated_cost_basis`, `realized_pnl`, `realized_return_rate`를 바꾸지 않고 이후 v2 거래를 허용한다.

열린 legacy lot의 자동 migration은 이 ADR에서 수행하지 않는다. 부분 SELL이 뒤따른 history는 BUY version만 바꾸면 이후 SELL 파생값이 달라지므로, migration 도구는 해당 lot의 모든 SELL 원가·손익·수익률을 순서대로 다시 계산하고 같은 directory의 temporary file을 flush/fsync한 뒤 atomic rename과 directory fsync를 완료해야 한다. v2 순수량보다 SELL 수량이 크거나 SELL base fee가 있으면 자동 변환하지 않고 수동 reconciliation을 요구한다.

2.3 Idempotency와 crash 복구

- `order_id`는 trade record의 idempotency key다. 같은 order ID와 동일 canonical 내용의 append는 no-op이다.
- 같은 order ID에 다른 내용이 오면 `ORDER_HISTORY_CONFLICT`로 실패하고 자동 덮어쓰지 않는다.

- append는 한 JSON line과 LF를 한 번에 쓰고 flush/fsync한 뒤 성공을 반환한다.
- 마지막 줄만 LF 없이 끝났고 JSON parsing이 실패하면 그 bytes를 `<history>.corrupt-<UTC timestamp>`에 보존한 뒤 마지막 정상 LF까지 truncate한다.
- backup file과 그 parent directory를 모두 fsync한 뒤에만 원본을 truncate한다. directory fsync가 실패하면 원본은 변경하지 않고 startup을 중단한다.
- 중간 줄이 malformed이거나 LF로 끝난 마지막 줄이 malformed이면 자동 복구하지 않고 `HISTORY_CORRUPTED`로 startup을 중단한다.
- 파일 없음은 빈 history다. permission, decoding과 fsync 오류는 빈 history로 숨기지 않는다.
- Phase 4 startup load/recovery 동안에는 bootstrap process 하나가 history 경로를 독점하며 다른 Repository instance/process가 append하지 않는다. Phase 8 writer는 같은 소유권을 유지하거나 concurrent writer를 허용하기 전에 OS file lock을 추가한다.

2.4 Fee 증거와 외부 매도 확장

v3는 기존 필드에 `fee_fills`를 추가해 BNB의 체결 시점 평가 근거를 보존한다. 각 fill과 aggregate의 주문 ID·수량·금액·수수료·체결 시각이 일치해야 한다. BNB 이외의 지원하지 않는 제3 수수료 asset은 기존 reconciliation 정책을 유지한다.

v4는 같은 fill 근거를 가진 외부 SELL 전용 record다. `exit_reason=EXTERNAL_MANUAL`, `market_price_at_decision=null`이며, 실제 거래소 원 요청량을 기록한다. 앱 주문에는 이 이유와 null 판단 시세를 허용하지 않는다. 기존 row를 덮어쓰지 않고 주문 ID 기준으로 한 번만 append한다. 상세 검증과 잔여 회계는 [ADR-007](#)을 따른다.

3. Position cost basis와 Performance 공식

3.1 Average-cost Position

v2 BUY 체결의 Position cost basis와 수량은 실제 자산 흐름을 기준으로 한다. quote asset으로 수수료를 낸 경우에는 기존과 같이 수수료를 취득원가에 더한다. base asset인 ETH로 수수료를 낸 경우에는 거래소의 `executed_quantity`와 Trade 기록은 gross 체결 사실로 보존하되, Position에는 `executed_quantity - fee_amount`만 더한다. 이때 ETH 수수료의 `fee_quote_amount`는 별도의 quote 출금이 아니므로 cost basis에 다시 더하지 않는다.

```
if buy_fee_asset == quote_asset:
    acquired_quantity = buy_executed_quantity
    acquisition_cost = buy_executed_amount + buy_fee_quote
else if buy_fee_asset == base_asset:
    acquired_quantity = buy_executed_quantity - buy_fee_amount
    acquisition_cost = buy_executed_amount

new_cost_basis = old_cost_basis + acquisition_cost
new_quantity = old_quantity + acquired_quantity
average_entry_price = new_cost_basis / new_quantity
```

`acquired_quantity <= 0`은 정상 체결로 보지 않고 reconciliation을 요구한다. SELL에서 base asset 수수료가 발생하면 gross 매도량 외의 추가 base depletion과 realized PnL 공식이 함께 필요하므로 Phase 9에서는 자동 반영하지 않고 fail closed한다. 현재 ETHUSDT의 일반적인 SELL quote-asset 수수료만 아래 공식을 적용한다.

v1 history replay에는 2.2의 legacy gross 수량·fee 포함 원가 공식을 적용하며, 이 분기는 신규 execution이나 v2 row에 사용하지 않는다.

SELL 직전의 수량을 `quantity_before`라 할 때 매도량의 원가는 다음과 같다.

```
allocated_cost_basis
= position_cost_basis * sell_executed_quantity / quantity_before
```

Position에 SELL을 적용하기 전에 이 값을 고정한다. 전량 매도 뒤 남는 미세 Decimal은 0으로 정규화하되 exchange step-size 범위와 테스트로 제한한다.

3.2 실현 결과

```
net_sell_proceeds = sell_executed_amount - sell_fee_quote
realized_pnl = net_sell_proceeds - allocated_cost_basis
realized_return_rate = realized_pnl / allocated_cost_basis * 100
```

`allocated_cost_basis == 0`은 정상 매도 결과가 아니며 수익률을 0으로 만들지 않는다.

`INVALID_ZERO_COST_BASIS`로 reconciliation을 요구한다. 계산 결과는 소수점 8자리 `ROUND_HALF_EVEN`으로 표현하고 원금/손익 금액은 exchange quote precision에서 별도로 표시 반올림한다.

3.3 집계

- KST 당일은 [00:00:00, 다음 날 00:00:00)의 반열린 구간이다.
- `daily_return_rate` = 당일 SELL realized_pnl 합 / 당일 SELL allocated_cost_basis 합 * 100. 당일 SELL이 없으면 0이다.
- `cumulative_return_rate`는 전체 SELL에 같은 공식을 적용한다. SELL이 없으면 0이다.
- `average_sell_return_rate`는 cost basis가 유효한 SELL별 수익률의 산술평균이다.
- `realized_pnl`과 `total_profit`은 fee 포함 전체 누적 실현손익이며 두 값은 동일하다.
- `daily_fee`는 KST 당일 BUY/SELL의 `fee_quote_amount` 합이다.
- `total_fee`는 전체 BUY/SELL의 `fee_quote_amount` 합이다.
- win은 `realized_pnl > 0`, loss는 `< 0`, breakeven은 `== 0`이다.
- completed sell count는 win + loss + breakeven이고, `win_rate = win / completed * 100`이다. completed가 0이면 wire 값은 null이다.

3.4 Performance golden example

1. 2 ETH를 100 USDT에 BUY하고 BUY fee가 0.20 USDT다.
2. 1 ETH를 110 USDT에 SELL하고 SELL fee가 0.11 USDT다.
3. 나머지 1 ETH를 90 USDT에 SELL하고 SELL fee가 0.09 USDT다.

값	기대 결과
최초 cost basis	200.20
첫 SELL allocated cost	100.10

값	기대 결과
첫 SELL realized PnL	9.79
첫 SELL return	9.78021978%
둘째 SELL allocated cost	100.10
둘째 SELL realized PnL	-10.19
둘째 SELL return	-10.17982018%
누적/같은 날 realized PnL	-0.40
누적/같은 날 return	-0.19980020%
평균 SELL return	-0.19980020%
win/loss/breakeven	1 / 1 / 0, win rate 50%
total/daily fee	0.40

KST 경계 예시는 다음과 같다.

- 2026-08-20T14:59:59Z는 KST 2026-08-20 23:59:59에 포함한다.
- 2026-08-20T15:00:00Z는 KST 2026-08-21 00:00:00에 포함한다.

4. History summary 범위

Trade History 화면의 table row만 period/side filter를 적용한다. 상단 summary는 filter와 무관하게 다음 authoritative 범위를 사용한다.

- 당일 수익률·당일 수수료: 현재 KST account day 전체
- 매도 성과·누적 실현손익: 현재 account의 전체 durable history
- ETH 보유량: 최신 Account/Position snapshot

UI label은 각각 오늘 계좌 성과, 전체 매도 성과, 현재 ETH 보유량처럼 범위를 명시해야 한다. 매수/매도 filter 변경으로 summary를 재계산하거나 filtered row 결과로 Account/Performance를 덮어쓰지 않는다.

4.1 거래 표의 진입 가격과 수수료 표시

진입당시 ETH가격은 사용자가 선택한 매수 주문의 `average_fill_price`다. BUY 행은 해당 매수의 실제 평균 체결가를, SELL 행은 동일 상품·REGIME·전략의 직전 BUY 평균 체결가를 표시한다. Controller는 필터 전 전체 이력에서 대응 가격을 연결하므로 기간 밖 매수나 매도 전용 필터에서도 값이 유지된다. 대응 매수 근거가 없으면 null이다. 주문 판단 시세 `market_price_at_decision`, 현재 시장가, 수수료를 포함한 원가 평단가로 대체하지 않는다. Trade JSONL은 수정하지 않는다.

개별 거래 수수료는 실제 `fee_amount`와 `fee_asset`을 우선 표시하며 유효 소수 자릿수를 자르지 않는다. ETH·BNB 수수료의 USDT 환산액은 `≈`로 구분해 보조 표시한다. 원 수수료가 USDT이면 원 금액만 표시한다. MIXED 수수료는 quote 환산액에 환산을 명시하고 자산별 원 수수료를 함께 표시한다. 계좌 요약 카드와 손익 계산은 기존 quote 환산 회계 기준을 유지한다.

5. CSV schema와 파일 정책

5.1 조회 범위와 column

CSV 시작일과 종료일은 KST LocalDate이며 양끝 날짜를 포함한다. 내부 UTC query는 `startDate 00:00:00+09:00` 이상, `endDate + 1일 00:00:00+09:00` 미만이다. side는 ALL이다.

schema version 1의 column 순서는 다음과 같다.

```
schema_version,trade_id,order_id,client_order_id,executed_at_utc,
executed_at_kst,symbol,side,regime_type,strategy,average_fill_price,
market_price_at_decision,executed_quantity,executed_amount,fee_amount,
fee_asset,fee_quote_amount,allocated_cost_basis,realized_pnl,
realized_return_rate,exit_reason
```

실제 header는 위 항목을 한 줄에 이어 쓴다. null 값은 빈 field다. Decimal은 JSONL과 같은 plain string, UTC는 RFC 3339 Z, KST는 offset이 포함된 ISO 8601로 기록한다.

5.2 생성 정책

- encoding은 UTF-8 with BOM이고 CSV row separator는 CRLF다.
- RFC 4180 quoting 규칙을 사용한다.
- 조회 결과가 0건이면 `NO_TRADES_TO_EXPORT`로 실패하며 파일과 temporary file을 만들지 않는다.
- `file_name`은 path separator와 traversal이 없는 basename이어야 한다. `.csv`가 없으면 한 번 붙이고, 대소문자와 무관하게 이미 있으면 중복 추가하지 않는다.
- `destination`이 이미 있으면 `DESTINATION_EXISTS`로 실패하며 overwrite하지 않는다.
- 같은 directory에 `.<final-name>.<uuid>.tmp`를 생성해 write → flush → fsync한 뒤 no-replace atomic rename을 수행한다.
- macOS/Linux는 native exclusive rename, Windows는 `destination`을 교체하지 않는 rename을 사용한다. rename 성공 뒤 지원되는 POSIX filesystem의 parent directory fsync는 best-effort로 보강하되 이미 게시된 결과를 불안정하게 rollback하지 않는다.
- 오류 시 temporary file은 best-effort로 제거하고 원래 `destination`은 건드리지 않는다.
- 성공 결과는 absolute path와 실제 data row count를 반환한다.

6. 검증 의무

- ☒ EMA9, LR slope, swing, live snapshot의 공식과 golden vector가 확정되었다.
- ☒ JSONL schema, Decimal/timestamp 표현과 crash 복구가 확정되었다.
- ☒ Performance 분모, fee, 승패와 KST 경계가 numeric example로 확정되었다.
- ☒ summary와 filtered row 범위가 분리되었다.
- ☒ CSV column, encoding, empty, overwrite와 atomic write가 확정되었다.
- ☐ Phase 3에서 indicator golden vector를 자동 테스트로 옮긴다.
- ☒ Phase 4에서 JSONL/Performance golden test를 구현했다.
- ☒ Phase 11에서 CSV golden file과 filesystem fault matrix를 구현했다.